Build a CNN for character recognition using the MNIST dataset

To build a Convolutional Neural Network for handwritten digit recognition using the MNIST dataset and evaluate its performance.

Algorithm

1. Import TensorFlow and the required Keras libraries.
2. Load the MNIST training and testing datasets.
3. Normalize the pixel values from 0–255 to 0–1.
4. Reshape the images to include a single grayscale channel.
5. Create a Sequential CNN model.
6. Add a convolutional layer with ReLU activation to extract image features.
7. Add a max-pooling layer to reduce image dimensions.
8. Add another convolutional and max-pooling layer.
9. Flatten the extracted feature maps into a one-dimensional vector.
10. Add a Dense layer with 64 neurons.
11. Add an output layer with 10 neurons using softmax activation.
12. Compile, train and evaluate the model using the MNIST test dataset.

Program

```
import tensorflow as tf
from tensorflow.keras import layers, models
tf.keras.utils.set_random_seed(10)
# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = (
tf.keras.datasets.mnist.load_data()
)
# Normalize images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
# Reshape images
x_train = x_train[..., None]
x_test = x_test[..., None]
# Create CNN model
model = models.Sequential([
layers.Input(shape=(28, 28, 1)),
layers.Conv2D(32, (3, 3), activation='relu'),
layers.MaxPooling2D((2, 2)),
layers.Conv2D(64, (3, 3), activation='relu'),
layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
])
# Compile model
model.compile(
optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)
# Train model
model.fit(
x_train, y_train,
epochs=3,
batch_size=128,
validation_split=0.1
)

# Evaluate model
loss, accuracy = model.evaluate(
x_test, y_test, verbose=0
)
print("Test Accuracy:", accuracy * 100, "%")
# Predict sample images
predictions = model.predict(x_test[:5], verbose=0)
print("Actual Labels:", y_test[:5])
print("Predicted Labels:", predictions.argmax(axis=1))
```

Output of Program

```
Epoch 1/3
Epoch 2/3
Epoch 3/3
Test Accuracy: Approximately 98%
Actual Labels:
[7 2 1 0 4]
Predicted Labels: [7 2 1 0 4]
```

Result

Thus, a CNN model was successfully implemented using the MNIST dataset. The model learned handwritten digit features and achieved high classification accuracy on the testing dataset.